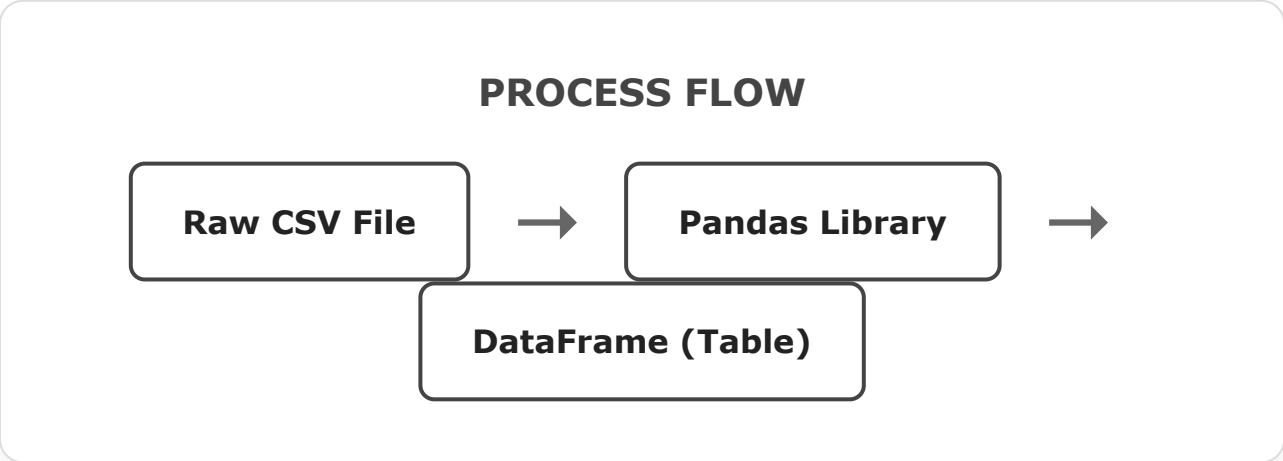
1. THE LOAN PREDICTION PROJECT

THE MISSION: "AUTOMATE THE BANK"

Imagine you are working at "SafeBank". Every day, people apply for loans. Currently, a human manager reads every application (Income, Credit Score, etc.) and decides: **YES** or **NO**.

This is slow! Your boss wants you to build an **AI Program** that learns from past data and makes these decisions automatically. Finally, you will turn this AI into a **Real Website** using Streamlit.

PHASE 1: LOADING THE DATA



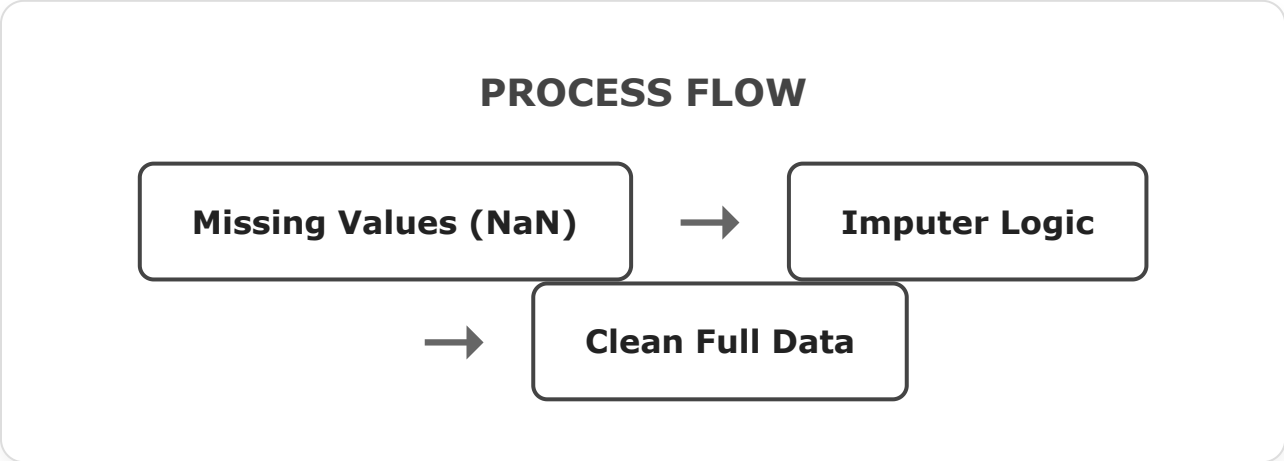
> 💡 **SIMPLE EXPLANATION:**
>
> *Think of a **CSV file** like an Excel sheet. Python can't read it directly, so we use a library called **Pandas** to load it into a table format called a DataFrame.*

STEP 1: WRITE THE CODE

PYTHON CODE

```python
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder,
MinMaxScaler

# Load the training and testing files from GitHub
raw_train =
pd.read_csv('https://raw.githubusercontent.com/bipulshah
i/loan_app/refs/heads/main/train_loan.csv')
raw_test =
pd.read_csv('https://raw.githubusercontent.com/bipulshah
i/loan_app/refs/heads/main/test_loan.csv')

# Create copies so we keep the original safe
train_df = raw_train.copy()
test_df = raw_test.copy()

# Look at the first 5 rows
train_df.head()
```

PHASE 2: CLEANING THE DATA (IMPUTATION)



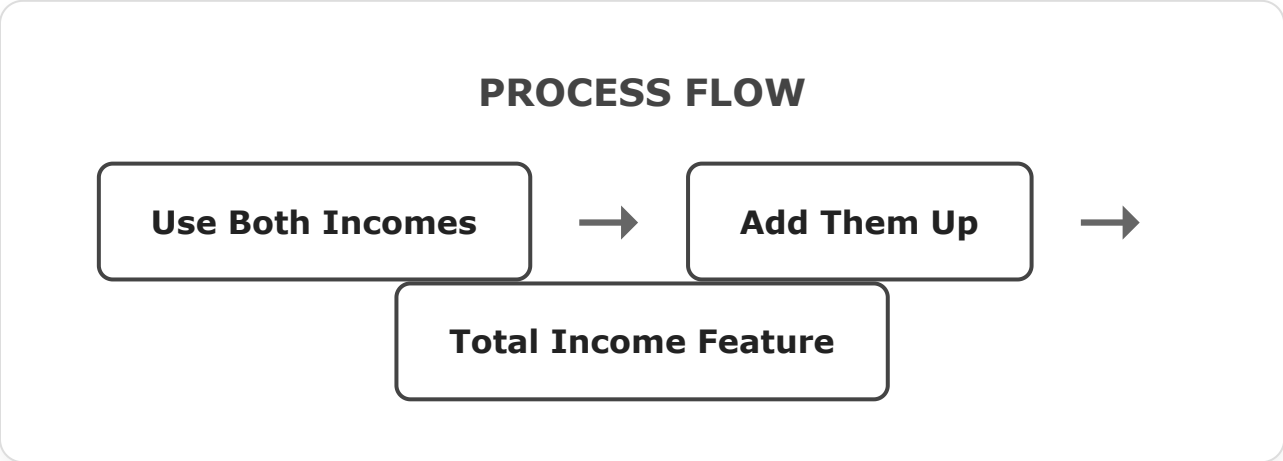
> 💡 **SIMPLE EXPLANATION:**
>
> *Real data is messy. Some rows have empty cells (NaN).*
>
> • *If a **Number** is missing (like Income), we fill it with the **Average (Mean)**.*
> • *If a **Category** is missing (like Gender), we fill it with the **Most Common (Mode)**.*

STEP 1: FILL THE MISSING VALUES

PYTHON CODE

```python
# 1. Separate Columns by Type
num_cols = ["ApplicantIncome", "CoapplicantIncome",
            "LoanAmount", "Loan_Amount_Term"]
cat_cols = ['Gender', 'Married', 'Dependents',
'Education',
            'Self_Employed', 'Credit_History',
'Property_Area']

# 2. Fill CATEGORIES with the Most Frequent value
cat_imputer = SimpleImputer(strategy="most_frequent")
cat_imputer.fit(train_df[cat_cols])
train_df[cat_cols] =
cat_imputer.transform(train_df[cat_cols])
test_df[cat_cols] =
cat_imputer.transform(test_df[cat_cols])

# 3. Fill NUMBERS with the Mean value
num_imputer = SimpleImputer(strategy="mean")
num_imputer.fit(train_df[num_cols])
train_df[num_cols] =
num_imputer.transform(train_df[num_cols])
test_df[num_cols] =
num_imputer.transform(test_df[num_cols])
```

PHASE 3: CREATING BETTER FEATURES



> 💡 **SIMPLE EXPLANATION:**
>
> *The bank cares about the **Family's Total Income**, not just one person's income. So, we add them together. Also, computers only understand numbers. We must convert text (Male/Female) into numbers (0/1).*

STEP 1: CREATE TOTAL INCOME

PYTHON CODE

```python
# Add incomes together
train_df['ApplicantIncome'] =
train_df['ApplicantIncome'] +
train_df['CoapplicantIncome']
test_df['ApplicantIncome'] = test_df['ApplicantIncome']
+ test_df['CoapplicantIncome']

# Remove the extra column
train_df = train_df.drop('CoapplicantIncome',
axis='columns')
test_df = test_df.drop('CoapplicantIncome',
axis='columns')

# Update our list of numeric columns
num_cols.remove('CoapplicantIncome')
```

STEP 2: TURN TEXT INTO NUMBERS (ENCODING)

PYTHON CODE

```python
# Loop through every text column and convert to numbers
for col in cat_cols:
    le = LabelEncoder()
    train_df[col] = le.fit_transform(train_df[col])
    test_df[col] = le.fit_transform(test_df[col])
```

PHASE 4: HANDLING HUGE NUMBERS (OUTLIERS)

PROCESS FLOW

Extremely High Income → **Log Transform** → **Normal Range Numbers**

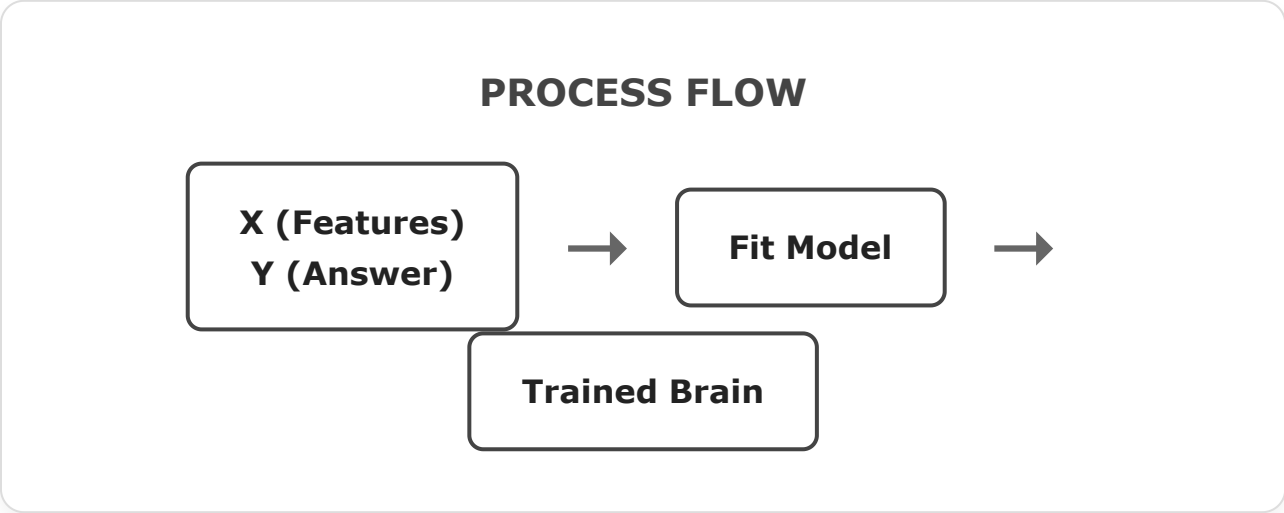
> 💡 **SIMPLE EXPLANATION:**
>
> *Some people are billionaires. If we use their raw income, it will confuse the model.* ***Log Transformation*** *shrinks huge numbers so they act like normal numbers. Then we squeeze all numbers between 0 and 1.*

STEP 1: APPLY LOG AND SCALE

PYTHON CODE

```python
# 1. Log Transform (Shrink outliers)
train_df[num_cols] = np.log(train_df[num_cols])
test_df[num_cols] = np.log(test_df[num_cols])

# 2. Scale (Squeeze everything between 0 and 1)
minmax = MinMaxScaler()
train_df = minmax.fit_transform(train_df)
test_df = minmax.transform(test_df)
```

PHASE 5: CREATING THE BRAIN (TRAINING)



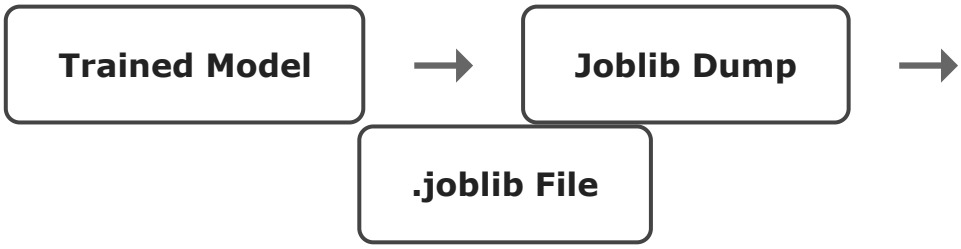
STEP 1: TRAIN THE MODEL

PYTHON CODE

```python
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Separate Target (Y) from Features (X)
# Note: Ensure y_train is ready (from
raw_train['Loan_Status'])

# Split data: 70% to learn, 30% to test
x_train, x_test, y_train, y_test = train_test_split(
    train_df, train_y, train_size=0.7, random_state=0
)

# Create the Brain
log = LogisticRegression()

# Teach the Brain
log.fit(x_train, y_train)

# Test the Brain
ytestPred = log.predict(x_test)
print("Accuracy:", accuracy_score(y_test, ytestPred))
```

PHASE 6: SAVING THE BRAIN (JOBLIB)

PROCESS FLOW



> 💡 **SIMPLE EXPLANATION:**
>
> *If we close Python, the brain forgets everything. We must save it to a file so we can load it later in our website.*

STEP 1: SAVE FILES

PYTHON CODE

```python
import joblib

# Save the Model
joblib.dump(log, 'modellog.joblib')

# Save the Scaler (Always save your scaler!)
joblib.dump(minmax, 'minmaxscaler.joblib')
```